

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA-0409

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

Experiment Questions:

1. Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.
2. Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.
3. Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.
4. Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.
5. Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Logical Address to Physical Address Using Base and Limit

Explanation

The **base register** stores the starting physical address and the **limit register** specifies the valid address range.

Physical Address = Base + Logical Address

C Program

```
#include <stdio.h>

int main()
{
    int base, limit, logical, physical;

    scanf("%d", &base);
    scanf("%d", &limit);
    scanf("%d", &logical);

    if (logical >= 0 && logical < limit)
    {
        physical = base + logical;
        printf("Valid Address\n");
        printf("Physical Address: %d\n", physical);
    }
}
```

```
else
{
    printf("Invalid Address\n");
}

return 0;
}
```

Sample Input

1000

500

250

Output

Valid Address

Physical Address: 1250

Result: Logical address is successfully translated into a physical address.

Q2. Paging Memory Management

Explanation

Paging divides memory into **pages and frames**. A page table maps pages to frames.

Physical Address = Frame Number × Page Size + Offset

C Program

```
#include <stdio.h>
```

```
int main()

{

    int n, size, page[100];

    int p, offset, physical, i;


    scanf("%d", &n);

    scanf("%d", &size);


    for (i = 0; i < n; i++)

        scanf("%d", &page[i]);


    scanf("%d", &p);

    scanf("%d", &offset);


    if (p >= 0 && p < n && offset >= 0 && offset < size)

    {

        physical = page[p] * size + offset;

        printf("Frame Number: %d\n", page[p]);

        printf("Physical Address: %d\n", physical);

    }

    else
```

```
printf("Invalid Address\n");
```

```
return 0;
```

```
}
```

Sample Input

4

100

3

1

5

2

2

50

Output

Frame Number: 5

Physical Address: 550

Result: Paging and address translation are successfully implemented.

Q3. Page Table Simulation

Explanation

A page table maps a **logical page number to a physical frame number**.

C Program

```
#include <stdio.h>
```

```

int main()

{
    int table[100], n, size, page, offset, i;


    scanf("%d", &n);

    scanf("%d", &size);


    for (i = 0; i < n; i++)

        scanf("%d", &table[i]);


    scanf("%d", &page);

    scanf("%d", &offset);


    if (page >= 0 && page < n && offset < size)
    {
        printf("Page %d -> Frame %d\n", page, table[page]);

        printf("Physical Address: %d\n",

            table[page] * size + offset);
    }
    else

        printf("Invalid Address\n");

```

```
    return 0;  
}
```

Sample Input

4
100
4
2
7
1
2
30

Output

Page 2 -> Frame 7

Physical Address: 730

Result: The page table successfully maps pages to frames.

Q4. Translation Lookaside Buffer (TLB)

Explanation

TLB is a **fast cache** that stores recently used page-table entries. It reduces address translation time.

Formula

$$\text{EAT} = \text{Hit Ratio} \times (\text{TLB} + \text{Memory}) + \text{Miss Ratio} \times (\text{TLB} + 2 \times \text{Memory})$$

C Program

```
#include <stdio.h>

int main()
{
    float tlb, memory, hit, miss, eat;

    scanf("%f", &tlb);
    scanf("%f", &memory);
    scanf("%f", &hit);

    miss = 1 - hit;

    eat = hit * (tlb + memory)
        + miss * (tlb + 2 * memory);

    printf("EAT with TLB: %.2f ns\n", eat);
    printf("Access Time without TLB: %.2f ns\n", memory);

    return 0;
}
```

Sample Input

10

100

0.8

Output

EAT with TLB: 130.00 ns

Access Time without TLB: 100.00 ns

Result: The program successfully demonstrates TLB and calculates effective access time.

Q5. Memory Protection Using Base and Limit

Explanation

Base and limit registers protect memory by checking whether the logical address is within the permitted range.

Valid if: $0 \leq \text{Logical Address} < \text{Limit}$

C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int base, limit, logical, physical;
```

```
    scanf("%d", &base);
```

```
    scanf("%d", &limit);
```

```
scanf("%d", &logical);

if (logical >= 0 && logical < limit)
{
    physical = base + logical;
    printf("Access Allowed\n");
    printf("Physical Address: %d\n", physical);
}
else
{
    printf("Access Denied\n");
}

return 0;
}
```

Sample Input

2000

500

300

Output

Access Allowed

Physical Address: 2300

Result: Unauthorized memory access is successfully prevented.